



UNIVERSITÀ DI PISA

Master programme in Data Science and Business Informatic
Data mining: Foundations

Alessia Delconte, Ilenia Fortuna, Giulia Mela

A.Y. 2024/2025

Contents

1	Data understanding and preparation	2
1.1	Dataset	2
1.2	Distribution of variables	3
1.3	Handling missing values	5
1.4	Feature selection	5
1.5	Outliers detection	6
1.6	Correlations before and after data preparation	6
2	Clustering	7
2.1	K-Means	8
2.2	DBSCAN	9
2.3	Hierarchical clustering	10
2.4	Final discussion	11
3	Classification	11
3.1	K-Nearest Neighbors	12
3.2	Naive Bayes	14
3.3	Decision Tree	15
4	Regression	17
4.1	Single e Multiple Regression	18
4.2	Multivariate Regression	18
5	Pattern Mining	19

1 Data understanding and preparation

1.1 Dataset

The IMDb dataset used in this project contains information on 16,431 forms of visual entertainment, such as movies, TV series, and more. Each piece of content is described by 23 variables that represent its essential characteristics. The following table lists the 23 variables in the dataset, with a brief description of each and its respective datatype.

Name	Description	Type
originalTitle	Original title, in the original language	object
rating	IMDB title rating class	object
startYear	Represents the release year of a title. In the case of TV Series, it is the series start year	int64
endYear	TV Series end year	object
runtimeMinutes	Primary runtime of the title, in minutes	object
awardWins	Number of awards the title won	float64
numVotes	Number of votes the title has received	int64
worstRating	Worst title rating	int64
bestRating	Best title rating	int64
totalImages	Total Number of Images for the title within the IMDb title page	int64
totalVideos	Total Number of Videos for the title within the IMDb title page	int64
totalCredits	Total Number of Credits for the title	int64
criticReviewsTotal	Total Number of Critic Reviews	int64
titleType	The type/format of the title (e.g. movie, short, tvseries, tvepisode, video, etc)	object
awardNominationsExcludeWins	Number of award nominations excluding wins	int64
canHaveEpisodes	Whether or not the title can have episodes	bool
isRatable	Whether or not the title can be rated by users	bool
isAdult	Whether or not the title is for adult. 0: non-adult title; 1: adult title	int64
numRegions	The regions number for this version of the title	int64
userReviewsTotal	Total Number of Users Reviews	int64
ratingCount	The total number of user ratings submitted for the title	int64
countryOfOrigin	The country where the title was primarily produced	object
genres	The genre(s) associated with the title (e.g., drama, comedy, action)	object

Table 1.1: Description of the features

Upon examining this table, several insights can be immediately inferred:

- **endYear** is a variable that does not provide additional information for the "film" content type. Furthermore, despite being a numerical value, it is represented as an object.
- **numVotes** and **ratingCount** are highly similar variables.
- **isAdult** is represented as an integer, yet it is, in fact, a boolean variable.
- **runtimeMinutes** is represented as categorical, but it actually expresses a numerical value.

Before proceeding to the detailed data analysis stages, descriptive statistics analysis provides a comprehensive and immediate overview of the main characteristics of each numerical variable in the dataset, including counts, means, standard deviations, minimums, and maximums.

1.2 Distribution of variables

feature	count	mean	std	min	max	median	mode
startYear	16431	1991,867	26,120	1878	2024	1997	2016
awardWins	13813	0,491	2,974	0,0	145,0	0,0	0,0
numVotes	16431	1492,153	20137,708	5,0	966565	36	8
worstRating	16431	1,0	0,0	1,0	1,0	1,0	1,0
bestRating	16431	10,0	0,0	10,0	10,0	10,0	10,0
totalImages	16431	11,477	74,254	0,0	3504	1,0	1
totalVideos	16431	0,271	3,115	258,0	3504	0,0	0
totalCredits	16431	61,341	174,017	0,0	15742,0	34,0	8
criticReviewsTotal	16431	2,785	15,411	0,0	533,0	0,0	0
awardNominationsExcludeWins	16431	0,558	3,955	0,0	197,0	0,0	0
isAdult	16431	0,025	0,158	0,0	1,0	0,0	0
numRegions	16431	3,549	5,854	1,0	69,0	1,0	1
userReviewsTotal	16431	7,228	66,495	0,0	5727,0	0,0	0
ratingCount	16431	1492,915	20145,393	5,0	967042,0	36,0	8

Table 1.2: Descriptive statistics

From the analysis of the descriptive statistics, it emerges that:

- **awardWins** is the only variable with missing values, which is somewhat peculiar; this will be examined in greater depth in Section 1.3.
- **numVotes** and **ratingCount**, as hypothesized in the previous section, exhibit a high degree of similarity, sharing the same mode and median. However, they display a slight difference in their mean values. We will examine the two variables more thoroughly using correlation (Section 1.6).
- **worstRating** and **bestRating** appear to be highly static and minimally informative variables.

An additional step that allows us to better understand our variables is graphical visualization.

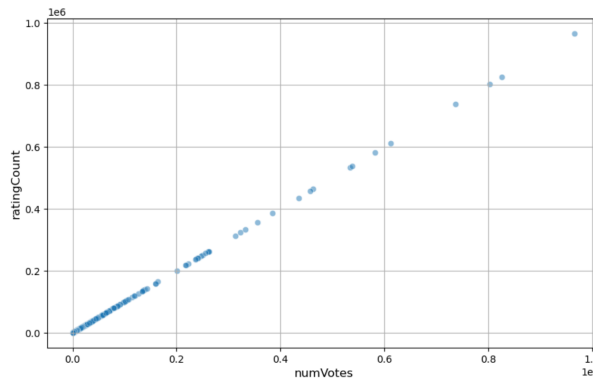


Figure 1.1: Graphical comparison between numVotes and ratingCount.

This chart (Fig 1.1) further confirms that the variables **numVotes** and **ratingCount** are nearly identical. The data points align almost perfectly along a straight line, indicating a very strong linear relationship and suggesting that the two variables are highly correlated (Section 1.6).

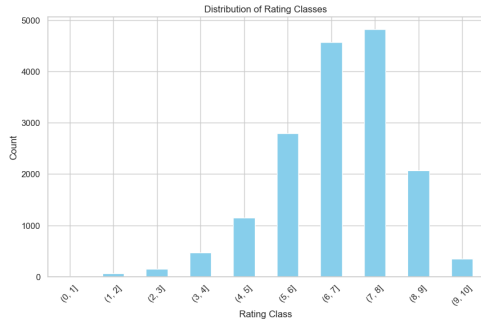


Figure 1.2: Distribution of Rating Classes

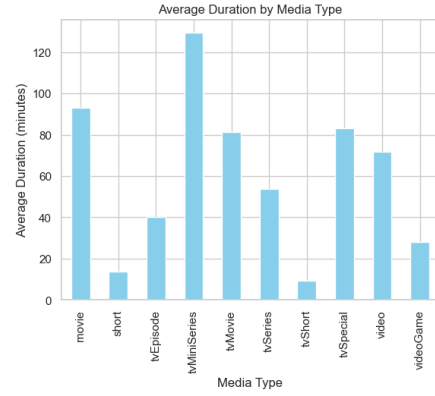


Figure 1.3: Average Duration by Media Type

The first graph (figure 1.2) shows a distribution that follows a bell-shaped pattern where the frequency peaks in the middle range of evaluation classes around 0.6-0.7 with approximately 4500 counts and gradually decreases towards both extremes of the scale from 0 to 1.0, showing lower frequencies at the beginning and end of the distribution range.

The second graph (figure 1.3) shows the average duration of various media types, using light blue bars. TV mini-series are the longest (130 minutes), followed by movies (90 minutes), while content like tvSpecial and video last 70-80 minutes. TV episodes last 40 minutes, video games 30 minutes, while short formats are the briefest with less than 20 minutes.

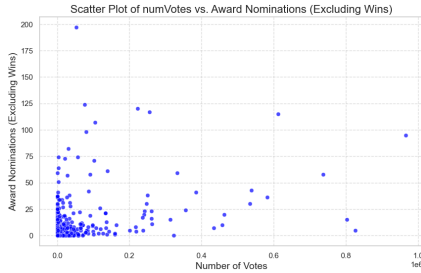


Figure 1.4: Scatter Plot of numVotes vs. Award Nominations excluding Wins

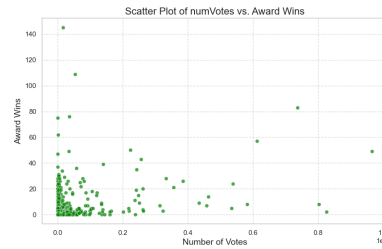


Figure 1.5: Scatter Plot of numVotes vs. Award Wins

The two scatter plots show the relationship between the number of votes and awards, but with different metrics:

- First plot shows a similar pattern, with most points concentrated on fewer votes. However, nominations go up to 200, suggesting that more items have many nominations compared to wins. Again, there is no clear correlation between votes and awards.
- Second plot shows that most data points have few votes and few wins (ranging from 0 to 20), with some outliers having more votes and wins (up to around 150). There is no clear correlation between the number of votes and wins.

Both plots show a concentration of items with few votes and few awards. The nominations plot has a wider range, with more items having high numbers of nominations, while the wins plot has a greater concentration of low awards. Overall, there is no strong correlation between votes and awards.

1.3 Handling missing values

Within the dataset, a significant number of missing values were found, originally marked as " $\backslash N$ ". Since this notation was not automatically interpreted as null, we converted these to NaN for correct data reading and handling. A heatmap was created to visualize the distribution of missing values.

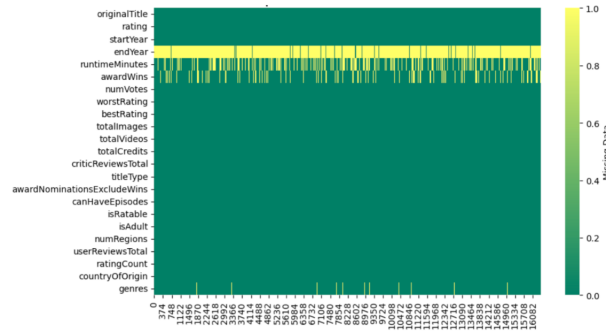


Figure 1.6: Heatmap of missing values

This graphic revealed the following variables with null values: **endYear** (15,617), **runtimeMinutes** (4,852), **awardWins** (2,618), and **genres** (382). Different methods were applied to manage these values based on the relevance and nature of each variable.

- **endYear**: With 95.05% of missing values (only 814 non-null out of 16,431), this variable was considered uninformative and, as detailed in section (Section 1.4), was excluded from subsequent analyses.
- **runtimeMinutes**: Given that 29.53% of values were missing, we replaced the missing data with the median, maintaining a more stable distribution without introducing biases.
- **awardWins**: For this variable, with 15.93% missing data, the missing values were replaced with the mode, which was considered an appropriate estimate to reflect the most common cases of awards won.
- **genres**: With only 2.32% of missing values, the missing data was replaced with "unknown" to maintain consistency without altering the categorization.

These choices were made to preserve the dataset's structure without introducing additional complexity.

1.4 Feature selection

The following modifications have been applied:

- Elimination of the **endYear** variable due to the high prevalence of missing values.
- Removal of **bestRating** and **worstRating** due to the low variance in their distribution.
- Exclusion of **ratingCount** after confirming its direct correlation with **numVotes**, rendering it redundant and not particularly informative.
- Transformation of **rating** from an interval scale to a mean value, now represented as a float.

- Exclusion of **totalImages** and **totalVideos**, with these variables being combined (along with those transformed in Section 1.5) into a new variable named **totalMediaContent**. By combining these two metrics, the analysis is streamlined, reducing dimensionality while maintaining the richness of the information related to the film’s media presence.
- The variable **isRatable** was removed as it consistently showed True across all 16,431 records in the dataset, so it isn’t informative.
- The variable **isAdult** was initially converted to a boolean and later removed because it duplicated the information already present in the genre column for the ‘adult’ category.

These adjustments were made in the pursuit of enhancing the dataset’s integrity and focusing on the most relevant features, ensuring clarity and precision in the analysis.

1.5 Outliers detection

From the analysis of the descriptive statistics of the numerical variables, we found that many of them exhibit a strongly skewed and peaked distribution, with a high concentration of values around zero and a long tail. This observation was confirmed by the visualization through box plots, particularly for the following variables: `awardWins`, `numVotes`, `totalImages`, `totalVideos`, `totalCredits`, and `userReviewsTotal`. Below, we provide a detailed account of the outliers detected for each variable:

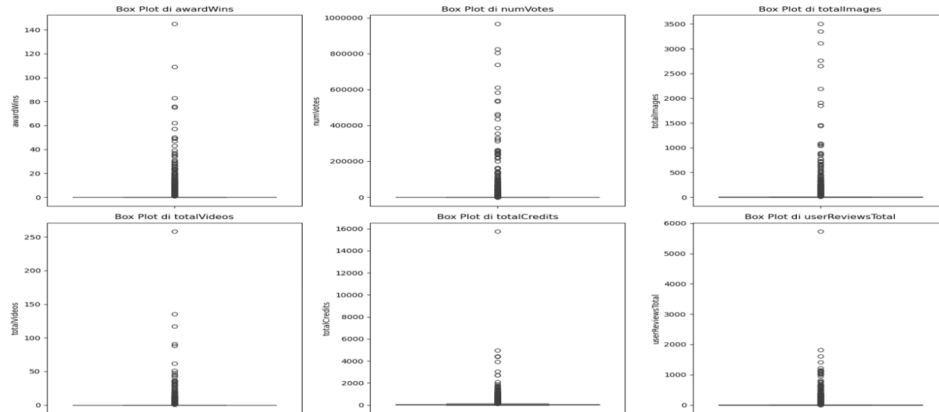


Figure 1.7: Boxplots visualization of outliers

The presence of numerous outliers, coupled with the significant distance between the upper whisker of the box plot and the outliers themselves, makes their removal challenging without compromising the integrity of the analysis. To address this issue, we applied a logarithmic transformation to `numVotes`, `totalImages`, `totalCredits`, and `userReviewsTotal`. This transformation reduced the impact of the outliers by shortening the distances and improving the overall distribution of the data, without eliminating potentially valuable information. For the variables `awardWins` and `totalVideos`, we chose to apply a square root transformation, which allowed for a more appropriate attenuation of the outlier distribution, given the specific nature and values of these variables.

1.6 Correlations before and after data preparation

We created a correlation matrix to examine the relationships between variables in the dataset. Figures 1.8 and 1.9 display the correlation at two different stages of the analysis. The first figure shows the correlation at the initial stage, while the second figure reflects the correlation after data understanding and preparation have been performed.

In the initial correlation matrix, we excluded **worstRating** and **bestRating** due to their constant values (1 and 10, respectively), which made them uninformative. We also identified redundancy between **ratingCount** and **numVotes**, as they were perfectly correlated, with **ratingCount** essentially being a derivative of **numVotes**. A high correlation between **startYear** and **endYear** (0.95) was observed, likely due to missing data for **endYear**. Additionally, **userReviewsTotal** and **ratingCount** showed a strong correlation (0.75), as they both reflect user interaction with the title.

In the second matrix, after transformations, there was a strong positive correlation between **numVotes_log** and **userReviewsTotal_log**, indicating consistent community engagement. Content ratings showed weak correlations with other variables, suggesting perceived quality is independent of factors like review volume or geographical coverage. Temporal variables like **startYear** and **runtimeMinutes** also had weak correlations with other indicators, indicating that success and engagement are not heavily influenced by these factors. Comparing the two matrices, the transformations (logarithmic and square root) reduced the correlations, particularly for variables with extreme or skewed distributions.

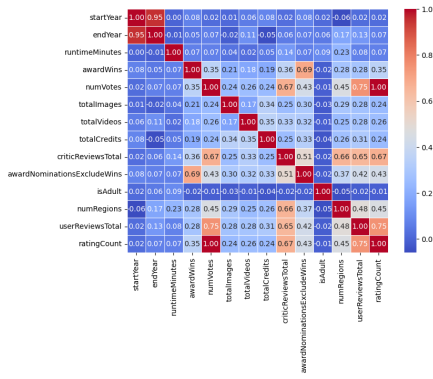


Figure 1.8: Correlation's heatmap before data understanding and data preparation

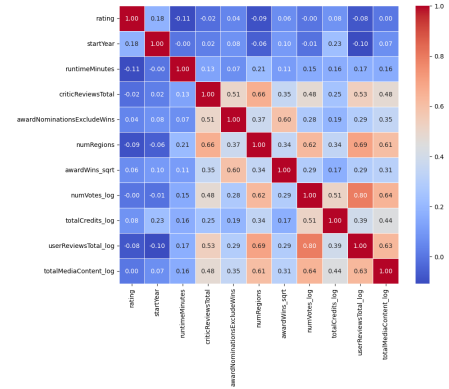


Figure 1.9: Correlation's heatmap after data understanding and data preparation

2 Clustering

This chapter examines the results obtained by applying clustering techniques to the dataset, using exclusively the available numerical variables.

The variables selected for the analysis are: **awardWins**, **numVotes**, **totalCredits**, **userReviewsTotal**, **totalMediaContent**, **runtimeMinutes**, **criticReviewsTotal**, **awardNominationsExcludeWins**, **startYear**, **numRegions**, and **rating** (NB: some of these have been transformed in section 1.5).

The high presence of outliers observed during the understanding phase highlighted the need to investigate the impact of these extreme values on the performance of clustering algorithms. So, the investigation was conducted on two versions of the dataset: one complete and the other without outliers, to show which algorithm performs better for each dataset.

We used Principal Component Analysis (PCA) to reduce the dimensionality of our dataset and simplify the clustering analysis. PCA allows us to preserve the most significant variance in the data by reducing the number of variables. We chose to use 4 principal components, as they together explain approximately 71.31% of the total variance, with the first component explaining 38.67% on its own.

2.1 K-Means

The *K-Means* algorithm is a centroid-based clustering method, where the goal is to partition the data into k distinct clusters. In this algorithm, each cluster is represented by a centroid, which is the mean of all the points in the cluster. The algorithm iterates by assigning each data point to the nearest centroid and then recalculating the centroids based on the points assigned to them.

To determine the optimal number of *clusters* (k), we used the *elbow method*, examining the behavior *SSE* and the *silhouette* score. Upon analyzing the results, we found that the most satisfactory k value for describing the structure of our dataset was 3.

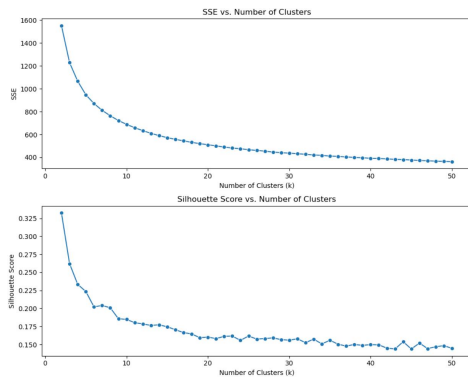


Figure 2.1: SSE and Silhouette of the full dataset

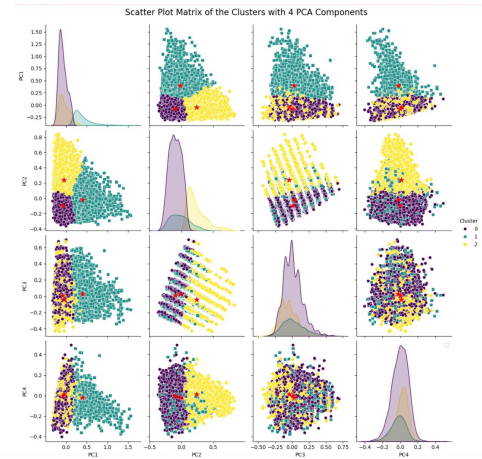


Figure 2.2: Scatter Plot Matrix of clusters of the full dataset

The *silhouette* score for this clustering was 0.26, while the *SSE* was 1228.53. These values suggest that, the clustering is reasonable, with a relatively good separation between clusters, though still improvable.

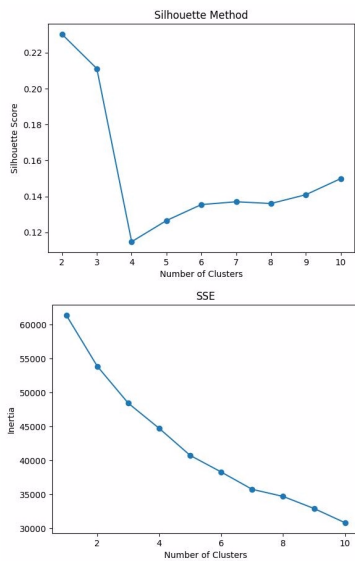


Figure 2.3: SSE and Silhouette of the dataset without outliers

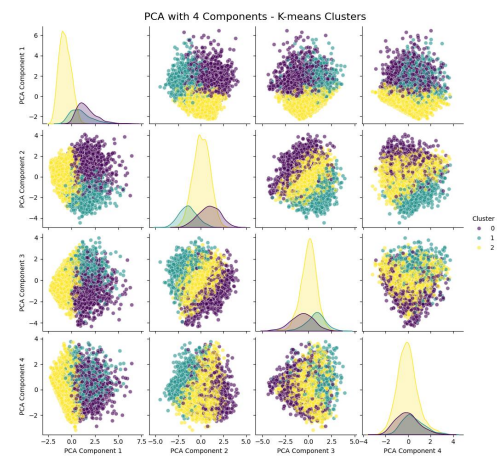


Figure 2.4: Scatter Plot Matrix of clusters of the dataset without outliers

We repeated the clustering on a dataset with outliers removed. The results showed an SSE of 48437.16 and a $silhouette$ score of 0.21. The SSE without outliers is significantly higher, and the $silhouette$ score is lower, indicating that the removal of outliers did not considerably improve the numerical measures of separation and compactness of the clusters. However, despite these altered metrics, the clusters in the dataset without outliers appear to be better distributed and more visually distinct, enhancing the interpretability of the clustering results.

These observations suggest that while the removal of outliers affects the SSE and $silhouette$ values, it improves the visual separation of clusters and does not hinder the clustering process. The presence of outliers, although influencing the SSE , did not negatively affect the overall structure of the clusters. The choice of 3 as the optimal number of clusters remains appropriate for describing the data structure effectively.

2.2 DBSCAN

For the clustering process with DBSCAN, which is a density-based clustering algorithm, selecting the appropriate parameters, namely eps and $min_samples$, was crucial. To determine these values, we employed the k -distance graph method. This technique involves plotting the distances of each point to its k -th nearest neighbor (in our case, the 4th nearest neighbor) and identifying the "elbow point" in the graph. So, we selected $eps = 0.23$, which corresponded to the point where the curve began to flatten, balancing the density requirements for cluster formation with the need to minimize noise.

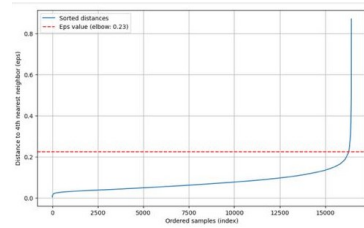


Figure 2.5: K-distance graph method

Next, several attempts were made to find the optimal value for $min_samples$. Initially, various values were tested, but we decided to focus on those that showed the most promising results based on the $silhouette$ score. After several trials, the combination of $eps = 0.23$ and $min_samples = 5$ provided the best clustering results, achieving a $silhouette$ score of 0.62 and SSE of 1859.1852

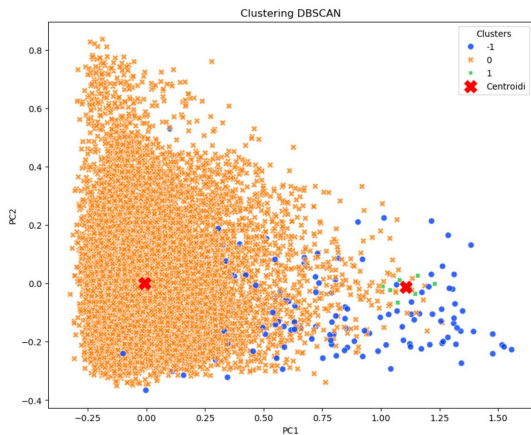


Figure 2.6: DBSCAN in PCA visualization for the full dataset

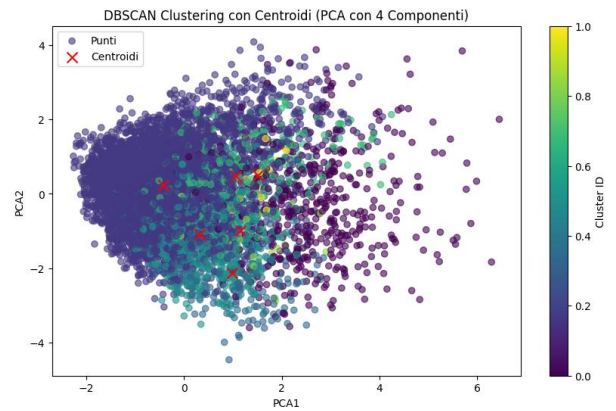


Figure 2.7: DBSCAN in PCA visualization for the dataset with outliers

When DBSCAN was applied to the dataset including outliers, the algorithm identified two clusters: *Cluster 0*: 16283 elements *Cluster 1*: 8 elements. The results indicate that the vast majority of data points are concentrated in a single, dominant cluster (Cluster 0), while a negligible proportion is grouped into a very small secondary cluster (Cluster 1). This suggests that the dataset, in its original form, exhibits a high degree of homogeneity, with limited variance to support the formation of multiple distinct clusters. The presence of such a significant size disparity between clusters can indicate one or more of the following:

- **Dominance of Noise or Outliers:** The dataset have a significant amount of noise, causing DBSCAN to label most points as part of a single cluster and isolate only a small number of data points into another cluster.
- **Lack of Well Defined Cluster Boundaries:** The chosen parameter values for *eps* and *min_samples*, while optimal based on silhouette scores and k-distance graphs, not fully capture the underlying structure of the dataset.

When the same algorithm was applied to the dataset after removing outliers, the results showed a higher degree of diversity in the clustering process. The clusters were distributed as follows: *Cluster -1 (Noise)*: 505 elements; *Cluster 0*: 5,043 elements; *Cluster 1*: 211 elements; *Cluster 2*: 675 elements; *Cluster 3*: 330 elements; *Cluster 4*: 37 elements; *Cluster 5*: 18 elements. The silhouette score for this configuration dropped to 0.14, which is significantly lower than the score obtained with the dataset containing outliers (0.62). This indicates that the removal of outliers, while resulting in more diverse clusters, compromised the compactness and separation of these clusters. Additionally, the SSE (alternative for DBSCAN) increased to 14,952.54, suggesting that the dispersion within the clusters was greater in the outlier-free dataset.

2.3 Hierarchical clustering

At the end we explored hierarchical clustering, an agglomerative method that begins by treating each data point as its own separate cluster. Step by step, the algorithm merges the closest pairs of clusters until only one remains. The proximity between clusters was calculated using different linkage methods, including Complete, Single, Average, and Ward. Each of these methods influences the structure of the resulting dendrogram.

In our analysis, we constructed dendrograms for each linkage method to visualize the clustering process. By carefully examining these dendrograms, we observed how data points were grouped at various levels, which provided valuable insights into potential cluster structures.

The **average linkage** method proved to be the most effective for clustering both the full dataset and the dataset without outliers. This method stood out because it produced well-separated clusters and achieved the highest silhouette score compared to the other methods.

The silhouette score measures how well each data point fits within its assigned cluster, with higher scores indicating more distinct and well-defined clusters. For the full dataset, the average linkage method yielded a silhouette score of 0.44, while for the dataset without outliers, the score was 0.43. These values were the highest among all the linkage methods, reinforcing the average linkage method as the optimal choice for creating coherent and well-separated clusters.

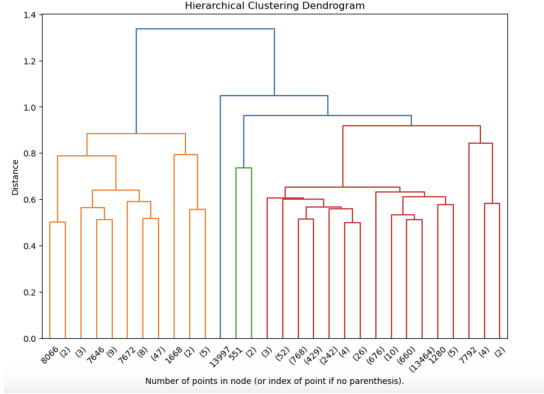


Figure 2.8: Average linkage hierarchical clustering dendrogram full dataset

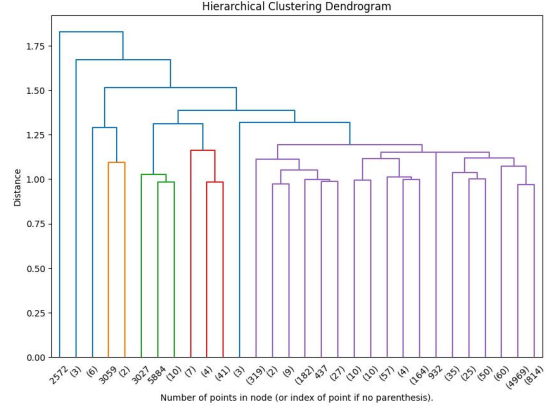


Figure 2.9: Average linkage hierarchical clustering dendrogram dataset without outliers

2.4 Final discussion

After conducting our analysis, we have reached the following conclusions: K-Means proved to be a good algorithm for clustering, with a Silhouette Score of 0.26 for the full dataset and an SSE of 1228.53. However, the clusters generated were not entirely distinct, suggesting some overlap between them, though it remains a valid option.

For DBSCAN, the selection of optimal parameters, `eps` = 0.23 and `min.samples` = 5, resulted in a silhouette score of 0.62 in the full dataset. However, the results for the dataset without outliers were less satisfactory, with a reduced Silhouette Score of 0.14 and a significant increase in SSE. This suggests that, although DBSCAN is suitable for datasets with varying densities, the removal of outliers compromised the quality of the segmentation.

Finally, the Average Linkage method showed superior results, with the highest Silhouette Score of 0.44 for the full dataset and 0.43 for the dataset without outliers, indicating that this method was the best at producing well-separated and coherent clusters. The structure of the dendrogram produced by Average Linkage, clear and easily interpretable, further reinforced its effectiveness compared to the other methods.

In conclusion, Average Linkage proved to be the best algorithm for our dataset, ensuring high-quality data segmentation both with and without outliers, thanks to its ability to generate well-defined and separated clusters. This method provided the most stable and interpretable results, with the highest silhouette scores compared to the other algorithms.

3 Classification

For the classification section of our project, we focused on applying three machine learning models: K-Nearest Neighbors (KNN), Naive Bayes, and Decision Tree, selecting the variables ‘genres’ and ‘titleType’ as the targets. For this phase, we used the provided test dataset, named ‘test.csv’, which contained 5478 rows. Before starting the analysis, we applied the same modifications previously made to the initial dataset (‘train.csv’) during the data understanding and data preparation phase, as described in Section 1. Additionally, to better adapt the dataset to the project’s needs, specific modifications were made to the target variables.

The ‘genres’ variable, initially characterized by multiple genres per content, was simplified. We decided to create four main classes based on a hierarchy already defined in the industry: ‘Comedy’, ‘Action’, ‘Drama’, and ‘Others’. Each genre in the dataset was assigned to one of these categories based on its thematic affinity, assigning the first genre listed for each content

to the corresponding macrocategory.

Similarly, the ‘titleType’ variable, which initially contained numerous values, some of which were quite similar, was divided into three main classes: ‘Movie’, ‘Series’, and ‘Short’.

Next, to avoid imbalances in the class frequencies, we balanced the different categories for both ‘genres’ and ‘titleType’, ensuring a more even distribution.

At the end, both target variables were encoded using one-hot encoding, creating a column for each class. In this way, each instance in the dataset was represented by 1 if the class was present and 0 if the class was not present.

For each model, before evaluating the performance on ‘test.csv’, we performed training on our training dataset. This resulted in the creation of two subsets: one for actual training and one for validation. The models were trained on these datasets, and various experiments were conducted to optimize performance using a variety of evaluation metrics. Once the best parameters were identified from the validation tests, the models were applied to the ‘test.csv’ dataset to verify whether they had successfully generalized the information learned.

3.1 K-Nearest Neighbors

Target: genres

A K-Nearest Neighbors (KNN) model was developed for the classification of the target variable **genres**.

Initially, a feature importance analysis was conducted using a Random Forest model. The results indicated that the most influential features for predicting **genres** showed an importance slightly above 10%, indicating relatively low relevance overall. The most impactful features were *totalCredits*, *startYear*, *runtimeMinutes*, and *numVotes*. Conversely, the least influential features included *awardNominationsExcludeWins*, *awardWins*, *criticReviewsTotal*, and the *titleType* variable, which was encoded into three classes.

Based on these findings, two distinct approaches were taken: training and evaluating the model using only the most important features, and separately, using all features, including the less relevant ones.

To optimize model performance, hyperparameter tuning was conducted using Grid Search, testing various values of k (ranging from 1 to 151), different distance metrics (Euclidean and Manhattan), and weight options (distance and uniform). The best results were obtained with the following parameters: With all features: $k = 59$, metric=Manhattan, weights=distance, achieving an accuracy of 0.49 in the validation test. With the most important features: $k = 33$, metric=Manhattan, weights=distance, achieving an accuracy of 0.47 in the validation test.

genres	Prec	Recall	F1	Supp
Action	0.51	0.68	0.58	1948
Comedy	0.61	0.29	0.40	1102
Drama	0.42	0.38	0.40	1402
Others	0.50	0.54	0.52	1026
Accuracy	0.50			
Macro avg	0.51	0.47	0.47	5478
Weighted avg	0.51	0.50	0.49	5478

Table 3.1: Results of the KNN for target genres with all variables (test set)

genres	Prec	Recall	F1	Supp
Action	0.49	0.63	0.55	1948
Comedy	0.49	0.27	0.35	1102
Drama	0.39	0.34	0.36	1402
Others	0.44	0.49	0.46	1026
Accuracy	0.46			
Macro avg	0.45	0.43	0.43	5478
Weighted avg	0.45	0.46	0.44	5478

Table 3.2: Results of the KNN for target genres with most important variables (test set)

Target genres: Analyzing the result

Analyzing the results obtained by the test it is possible to observe that both models show signs

of underfitting.

In the model using all the features, the accuracy is 50%, with precision and recall scores varying significantly across genres. Specifically, for the "Action" genre, the precision (0.51) and recall (0.68) are relatively good, while for the "Comedy" and "Drama" genres, the values are much lower. This suggests that the model fails to generalize properly for some genres, which is a clear sign of underfitting, meaning the model is not sufficiently complex to capture the variability in the data and, consequently, does not produce good results for all classes.

In the model using only the most relevant features, the accuracy further decreases to 0.46. Although the performance for "Action" remains relatively high, the scores for the other genres continue to be unsatisfactory. In this case, the reduction in features seems to have limited the model's ability to adapt properly to the data, leading to a further drop in performance. This suggests that by reducing the number of variables, the model lost crucial information needed for accurate predictions, resulting in insufficient generalization.

Target: titleType

For the analysis of the **titleType** target, the same procedures applied to the genres variable were followed. The most significant features were *runtimeMinutes*, *totalCredits*, and *startYear*, each demonstrating an importance around 10%. In contrast, the genres classes were found to be less relevant. In the model that incorporated all variables, the optimal result was achieved with: $k=9$, $\text{metric}=\text{Manhattan}$ and $\text{weights}=\text{distance}$, yielding an accuracy of 0.82 in the validation test. In the model with the most important features, the best result was obtained with $k=17$, again using the Manhattan metric and distance weighting, resulting in an accuracy of 0.80 in the validation test.

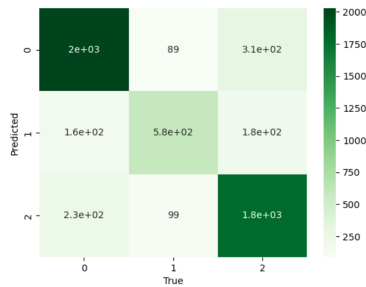


Figure 3.1: Confusion Matrix for model with important features

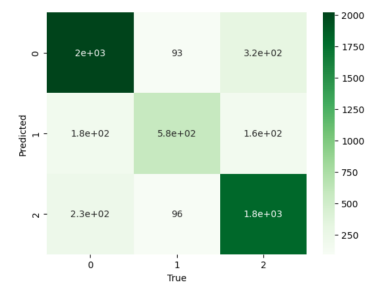


Figure 3.2: Confusion Matrix for model with all the features

titleType	Prec	Recall	F1	Supp
Movie	0.84	0.83	0.84	2444
Short Film	0.76	0.64	0.69	931
TV Series	0.81	0.87	0.84	2489
Accuracy	0.82			
Macro avg	0.80	0.78	0.79	5864
Weighted avg	0.82	0.82	0.81	5864

Table 3.3: Results of the classification for titleType with most important variables(testset)

titleType	Prec	Recall	F1	Supp
Movie	0.84	0.83	0.83	2444
Short Film	0.75	0.65	0.70	931
TV Series	0.82	0.87	0.84	2489
Accuracy	0.82			
Macro avg	0.80	0.78	0.79	5864
Weighted avg	0.82	0.82	0.82	5864

Table 3.4: Results of the classification for titleType with all variables (testset)

Target titleType: Analyzing the result

Both models show strong performance for Movies and TV Series. However, the Short Film class poses challenges, with lower precision (0.75-0.76) and recall (0.64-0.65), leading to more

misclassifications. The model using all features performs slightly worse for Movies but similarly for the other classes, indicating that including all features does not significantly enhance accuracy compared to using only the most important ones. This highlights the value of feature selection for maintaining efficiency without sacrificing performance.

3.2 Naive Bayes

In this analysis, we applied the Naive Bayes model to two different classification targets: *genres* and *titleType*. The performance was evaluated on both validation data and an external test set. Results were compared between using all features and only the important features, selected through a Random Forest model.

Target Genres

genres	Prec	Recall	F1	Supp
Action	0.62	0.18	0.28	1948
Comedy	0.26	0.04	0.07	1102
Drama	0.45	0.08	0.14	1402
Others	0.22	0.94	0.35	1026
Accuracy	0.27			
Macro avg	0.39	0.31	0.21	5478
Weighted avg	0.36	0.27	0.23	5478

Table 3.5: Results of Naive Bayes for target genres with all variables

genres	Prec	Recall	F1	Supp
Action	0.43	0.69	0.53	1948
Comedy	0.30	0.04	0.07	1102
Drama	0.42	0.17	0.25	1402
Others	0.35	0.55	0.43	1026
Accuracy	0.40			
Macro avg	0.38	0.36	0.32	5478
Weighted avg	0.40	0.40	0.35	5478

Table 3.6: Results of Naive Bayes for target genres with most important variables

The analysis of the results of Naive Bayes with all variables highlights generally low performance, with particularly limited precision and recall values for the *comedy* and *drama* classes. The *others* class, however, exhibits a very high recall (0.94), suggesting that the model has a good ability to identify this class. However, this comes at the expense of precision and F1-score. Overall accuracy is low (27%), reflecting a general difficulty in correctly discriminating between the various categories.

By selecting only the most important features, the model’s performance improves significantly. In particular, the *action* class shows a marked improvement in recall (0.69) and F1-score, reflecting the model’s enhanced ability to correctly identify this class. The *others* class also shows an increase in both precision and recall. However, the *comedy* and *drama* classes continue to exhibit unsatisfactory performance.

Target titleType

type	Prec	Recall	F1	Supp
Movie	0.87	0.24	0.37	2426
Short Film	0.22	0.92	0.36	925
TV Series	0.48	0.23	0.31	2127
Accuracy	0.35			
Macro avg	0.52	0.46	0.35	5478
Weighted avg	0.56	0.35	0.37	5478

Table 3.7: Results of Naive Bayes for title type with all variables

type	Prec	Recall	F1	Supp
Movie	0.80	0.55	0.65	2426
Short Film	0.45	0.68	0.54	925
TV Series	0.59	0.66	0.62	2127
Accuracy	0.62			
Macro avg	0.61	0.62	0.60	5478
Weighted avg	0.62	0.62	0.62	5478

Table 3.8: Results of Naive Bayes for title type with most important variables

The model exhibits high precision (0.87) for the *movie* class but a very low recall (0.24). This indicates that while the model can correctly identify *movie* when predicted, it struggles to correctly identify them in the dataset. The *shortFilm* class has a very high recall (0.92) but low precision (0.22), suggesting that the model frequently predicts *shortFilm* incorrectly for other categories. The *TV Series* class shows intermediate performance, with an F1-score of 0.31.

Using only the important features, the model shows significant improvement in all metrics. The *movie* class shows a notable increase in both precision and recall, achieving an F1-score of 0.65. The *shortFilm* class benefits from better balance between precision and recall, and "TV Series" exhibits more stable overall performance. The model's overall accuracy increased to 62%, highlighting the effectiveness of feature selection in improving performance.

Analyzing the results

In general, the application of the Naive Bayes model produced limited performance on both classifications, particularly for less represented classes such as *comedy* and *drama* (for genre classification) and *shortFilm* (for title type classification). However, the analysis highlighted that the selection of important features, based on the Random Forest model results, had a significant positive impact.

Key improvements observed include:

- An increase in overall accuracy, particularly for title type classification, where it improved from 35% to 62%.
- Evident improvements in the balance between precision and recall, with greater stability in predictions.
- Significant increases for the *action*, *movie*, and *tvSeries* classes, with higher F1-scores compared to the model using all features.

This analysis highlighted some of the fundamental limitations of the Naive Bayes model. In particular, the model tends to make simplifying assumptions about the distribution of data and the independence between variables, which makes it less suitable for complex datasets with correlated features. Additionally, the results showed poor performance in handling imbalanced classes, with particularly limited performance for underrepresented categories, such as 'Comedy' and 'Drama' in genre classification, or 'Short Film' in title type classification.

These drawbacks suggest that the model may not be ideal for applications where the data is highly heterogeneous or has asymmetric class distributions.

3.3 Decision Tree

The Decision Tree model was used as the final model to evaluate performance for both the *genres* target and the *titleType* target. The construction of the model was carried out in two main phases: initially, we used all the numeric features and categorical variables that we considered appropriate after properly encoding them. After reviewing how it performed and the results it produced, we moved on to the second phase: building the model based on the features that the model considered most important (for target *genres* were rating, startYear, runtimeMinutes, awardWins, numVotes, totalCredits, totalMediaContent and for target *titleType* were runtimeMinutes, totalCredits, numRegions, startYear, awardWins)

To improve the performance of the Decision Tree model, we applied several parameter optimization techniques, aiming to find the optimal configuration for each target. We optimized some hyperparameters of the Decision Tree model using *cross-validation*. Specifically, we tested different values for the `max_depth` of the tree, the `min_samples_leaf`, and the `min_samples_split`. For each parameter, we performed *5-fold cross-validation*, evaluating the average accuracy and standard deviation. The results were then plotted on graphs to identify the best configurations and optimize model performance.

We used also *RandomizedSearchCV* to optimize the hyperparameters of the Decision Tree model, exploring different combinations of values for `max_depth` (ranging from 2 to 20), `min_samples_split` (from 2 to 100), `min_samples_leaf` (from 1 to 100), and `criterion` (either *gini* or *entropy*), adjusting the number of iterations to balance performance and computation time. The final result

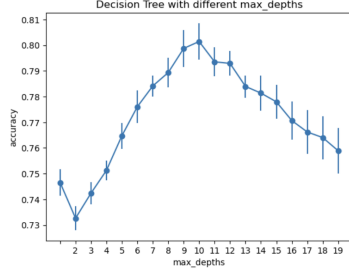


Figure 3.3: Graph for the maximum depth

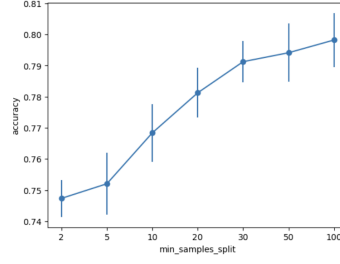


Figure 3.4: Graph for the minimum number of splits

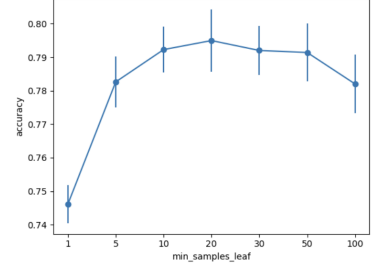


Figure 3.5: Graph for the minimum number of samples for a leaf

yielded the following parameters: `max_depth=9`, `min_samples_split=10`, `min_samples_leaf= 5`, and `criterion='entropy'`, which were then used for the final model.

We conducted numerous experiments to determine whether it was possible to further improve the model's performance while avoiding the risk of overfitting.

Target Genres: Analyzing the Results

The model with all the variables achieved higher accuracy (0.46) compared to the one using only the most important variables (0.42). Although the overall accuracy remains low, this difference highlights the importance of the removed variables, particularly for underrepresented classes like Comedy. For Comedy, precision was higher with all the variables (0.42 vs. 0.34), while recall slightly improved with the most important variables. This indicates that removing some variables reduced the model's ability to identify all instances of this class (lower recall) but led to fewer false positives (higher precision). The performance loss in Comedy and other minor categories suggests that the removed variables were critical in improving classification balance across classes, emphasizing their role in distinguishing between underrepresented groups.

genres	Prec	Recall	F1	Supp
Action	0.51	0.59	0.54	1741
Comedy	0.42	0.34	0.37	941
Drama	0.39	0.43	0.41	1257
Others	0.52	0.41	0.46	991
Accuracy	0.46			
Macro avg	0.46	0.44	0.45	4930
Weighted avg	0.46	0.46	0.46	4930

Table 3.9: Results of the DT for target genres with all variables

genres	Prec	Recall	F1	Supp
Action	0.54	0.43	0.48	1948
Comedy	0.34	0.44	0.38	1102
Drama	0.38	0.36	0.37	1402
Others	0.41	0.46	0.44	1026
Accuracy	0.42			
Macro avg	0.42	0.42	0.42	5478
Weighted avg	0.43	0.42	0.42	5478

Table 3.10: Results of the DT for target genres with most important variables

Target titleType: Analyzing the results

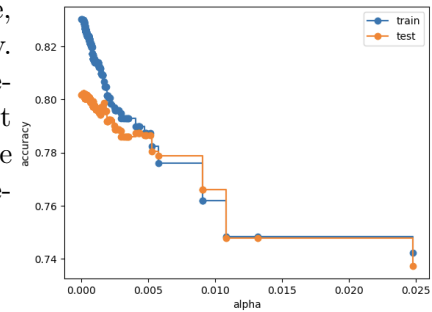
The Decision Tree models with all variables and with reduced variables showed similar accuracy (0.79 vs. 0.78), indicating that reducing the number of variables had minimal impact on overall performance. However, differences between the classes emerged.

For the Movie class, precision decreased slightly in the reduced model (0.85 vs. 0.80), while recall improved (0.83 vs. 0.85). This suggests that the reduced model was better at identifying more instances of the Movie class (higher recall), but at the cost of a slightly lower precision. For the Short class, precision increased from 0.63 to 0.74, while recall decreased from 0.76 to 0.64. This implies that the reduced model prioritized higher precision in classifying the Short class but sacrificed its ability to detect all instances of this class (lower recall). For the Series class, precision and recall metrics remained stable (precision 0.82 vs. 0.79 and recall 0.78 vs. 0.79), indicating that reducing the variables had no significant impact on this class. In general, reducing the number of variables resulted in improvements in certain areas, such as the Short class, but had minimal impact on the overall performance of the model.

titleType	Prec	Recall	F1	Supp
Movie	0.80	0.85	0.82	2426
Short	0.74	0.64	0.69	925
Series	0.79	0.79	0.79	2127
Accuracy	0.78			
Macro avg	0.78	0.76	0.77	5478
Weighted avg	0.79	0.79	0.79	5478

Table 3.7 Results of the DT for target titletype with the most important variables.

This chart illustrates how pruning impacts the decision tree, with the parameter **alpha** controlling the model's complexity. As **alpha** increases, the number of nodes and the depth decrease, simplifying the tree and reducing overfitting. The test accuracy reaches its peak at an optimal value of **alpha**, while very low or high values lead to overfitting or underfitting, respectively.



The second chart demonstrates the model's performance through ROC and Precision-Recall curves. The ROC curves show a high discriminative ability (AUC > 0.9 for all classes), with the **Movie** class achieving the best results. The Precision-Recall curves further confirm the model's strong performance, though the **Short** class appears to be more challenging to classify.

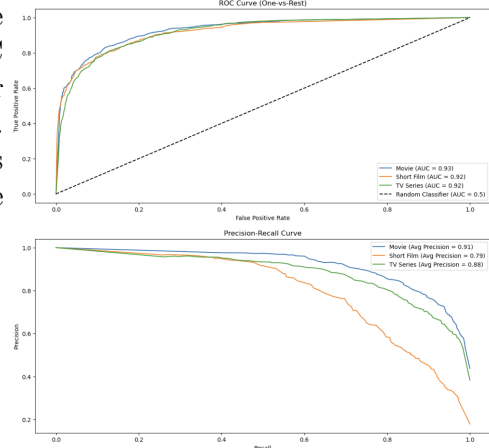


Figure 3.7 ROC and Prec-Rec Curve

4 Regression

In this analysis, we used the rating variable as the target in order to improve the prediction quality. Initially, our dataset contained a "rating" column with values represented as averages of rating intervals (e.g., 7.5, corresponding to a range of values). To improve the model's precision, we merged our training and test datasets with a dataset containing continuous and precise rating values, associating each value with the corresponding movie title. As a result, we replaced the average ratings with the precise ones. The preprocessing steps performed earlier remained unchanged during this phase.

4.1 Single e Multiple Regression

For the analysis of single and multiple regression using the target rating, several steps were performed to identify the best models and evaluate their performance. Regarding single regression, the correlation between rating and other variables was initially observed to determine which could serve as predictor. From this analysis, the best variables were startYear and totalCredits, although their correlation with rating was slightly above 0.15. Given the weak correlation, unsatisfactory results were anticipated, a hypothesis confirmed by the analysis of linear regression, Ridge, and Lasso, which showed no significant improvements. Following these results, nonlinear regression analysis was conducted, not limiting the predictors to startYear and totalCredits but testing the regression with each of the other features in the dataset. Nonlinear regression was calculated using two models: KNN and Decision Tree. Each predictor tested yielded unsatisfactory results, leading to the decision to optimize the models through Random Search. After optimization, the results showed marginal improvements. For the variable runtimeMinutes, the optimized Decision Tree demonstrated to be the most effective model compared to the others (see Table 4.1). It achieved the highest R^2 value (0.073), indicating that it explains a small but significant portion of the variance. Additionally, it had the lowest MSE (1.642) and the smallest MAE (0.961), confirming that it provides predictions with less error than other approaches. Conversely, KNN, both in its default and optimized versions, showed worse R^2 values (in one case even negative), with higher MSE and MAE, indicating inferior performance. For these reasons, the optimized Decision Tree was chosen as it better balances predictive capacity and low error levels.

Regression	Model	Target	R^2	MSE	MAE
Single	Decision Tree Optimized	runtimeMinutes	0.0727	1.6424	0.9612
Multiple	KNN Optimized	all features	0.2057	1.4069	0.8691

Table 4.1: Model Performance Metrics

Regarding multiple regression, the results showed significant improvements compared to single regression. Again, linear regression was initially analyzed, which did not yield satisfactory results. Subsequently, nonlinear regression was performed using Decision Tree and KNN, considering both all available features and a subset of the most important features. The best results were obtained with the optimized KNN model, which stood out for its superior performance compared to other methods analyzed (see Table 4.1). Specifically, the R^2 value (0.206) indicates that the model explains a larger portion of the variance in the data, albeit limited. The MSE (1.407) was the lowest among all tested models, while the MAE (0.869) confirmed a lower average absolute error compared to other approaches. These results suggest that the optimized KNN offers greater accuracy in predicting the target rating compared to the optimized Decision Tree, which comes close in performance but remains slightly inferior.

4.2 Multivariate Regression

To select the second variable to pair with *rating* for multivariate regression, we conducted several experiments to identify the variable that not only made logical sense but also yielded the most promising results in terms of model performance. After evaluating multiple candidates, we chose *totalCredits* as the most suitable option, although the final results were not exceptional overall. We initially tested linear regression models, but they did not perform well due to the complex relationships between the variables. As a result, we shifted our focus to non-linear models like the DT Regressor and KNN Regressor. At first, we used all available features as predictors to ensure a comprehensive evaluation. However, after assessing the models, we selected the most important ones based on the feature importance scores from the Decision Tree (DT) and

K-Nearest Neighbors (KNN) models. These models were better suited to capture the intricate interactions between the variables. We evaluated both the default and optimized versions of them, applying RandomizedSearch for hyperparameter tuning to improve performance. After training the models on the validation set and testing them on the test set, the optimized Decision Tree model emerged as the best performer for both target variables, providing the most accurate and reliable results. The best performance metrics from these experiments are summarized below.

Model	Target	R^2	MSE	MAE
DT Optimized	totalCredits	0.4879	0.6436	0.5759
DT Optimized	rating	0.1788	1.3413	0.8332
DT Optimized	avg of the two	0.3334	0.9924	0.7045

Table 4.2: Model Performance Metrics

The optimized Decision Tree model performed better in predicting *totalCredits* compared to *rating*. The R^2 for *totalCredits* was significantly higher than the one for *rating*, indicating a better explanation of the variability in the data. Also the MSE and the MAE seems to be better for totalCredits, respect to the ones for rating, suggesting that the model struggled more with making accurate predictions for this variable. When combining both targets, the average metrics stabilized at an intermediate value, with an R^2 of 0.3334, MSE of 0.9924, and MAE of 0.7045, highlighting the need for further model refinement to improve overall predictions.

5 Pattern Mining

Before applying pattern mining techniques such as the Apriori algorithm, it is crucial to pre-process the data to ensure that it is in the right format for analysis. This step is essential to enhance the quality of results and to avoid negative impacts from improperly prepared data. In this case, the preprocessing was carried out in several stages.

Data Preprocessing Steps

In the provided code, various data transformations were made to facilitate analysis, such as dividing numerical variables into categories. The values in the **rating** column were classified as *high_rating*, *medium_rating*, and *low_rating*, while the film duration (**runtimeMinutes**) was categorized as *short*, *medium*, or *long*. Similarly, the votes (**numVotes_log**) were divided into *few_votes*, *medium_votes*, and *many_votes*. Furthermore, boolean variables related to genres and title types were transformed into *True* or *False* values to represent the presence or absence of specific genres and types.

For association rule analysis, transactions were created for each film, including relevant genres and title types, as well as the start year categorized by decades. Information on nominations and awards was classified, and media content was categorized as *Low media content*, *Medium media content* or *High media content*. Additionally, the regions where the content was available were classified as *limited regions* or *many regions*. Finally, all this information was used to construct a one-hot matrix, where each column represents an attribute and each row represents a transaction, with binary values indicating the presence or absence of each attribute.

Frequent Itemsets Analysis

In the Apriori algorithm, the maximum support threshold, **max_supp**, and the minimum size for itemsets, **zmin**, are crucial parameters. In this case, we set:

$$\text{max_supp} = 25\% \quad \text{and} \quad \text{zmin} = 2$$

The graph shows the number of closed itemsets found at different levels of minimum support. The observations based on this graph are as follows:

- At low support levels (e.g., below 5%), the number of closed itemsets is high because many redundant subsets are included (smaller itemsets that belong to larger supersets).
- As the minimum support increases (e.g., above 10-15%), the number of closed itemsets decreases, as fewer combinations meet the support threshold.

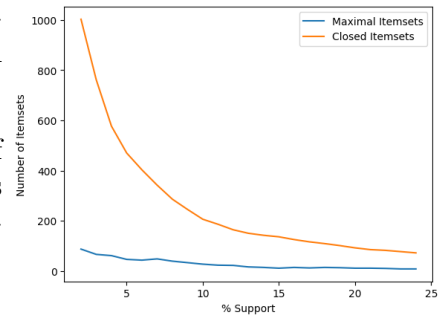


Figure 5.1: Maximal vs Closed Itemsets

The graph also shows the trend of maximal itemsets (blue line), which represent the itemsets that do not have any frequent supersets. The observations are:

- Maximal itemsets are always fewer in number compared to closed itemsets because they represent boundary cases.
- The number of maximal itemsets stabilizes quickly (after 10-15% support), indicating that beyond a certain threshold, only a few itemsets remain significant and have no supersets.

The curve representing closed itemsets is always above the one representing maximal itemsets, as all maximal itemsets are closed but not all closed itemsets are maximal. At low support, there is a significant difference between the two curves, as many closed itemsets are redundant and not maximal. At higher support, the difference decreases as redundancy reduces. For low support (less than 5%), too many itemsets are found, many of which are redundant, and the support threshold is too permissive, possibly not providing useful insights. In the intermediate support range (5-10%), there is a good balance between the quantity of itemsets and their significance, allowing the discovery of many interesting patterns. At high support (greater than 15%), only a few itemsets remain frequent, which is useful for identifying the most common patterns in the data, but it might overlook interesting but less frequent associations.

Frequent Itemsets and Association Rules

After running the Apriori algorithm, we identified the frequent itemsets and generated association rules. The following table shows the frequent itemsets found with their corresponding support values and lengths:

Support	Itemsets	Length
0.166454	Few nominations, 1990s	2
0.215507	2010s, Few nominations	2
0.168158	2010s, Limited regions	2
0.171444	No awards, Limited regions, Low media content, medium_votes, medium_rating	5
0.208447	No awards, Few nominations, Low media content, medium_votes, medium_rating	5

Table 5.1: Some Frequent Itemsets

The data shows combinations of features that frequently appear in the analyzed content. The patterns indicate that many contents from the 2010s have few nominations, limited geographic distribution, and average ratings. In particular, there are content items without awards and low media visibility that receive average votes and have similar evaluations. These patterns occur with support between 16% and 21%, suggesting that there are contents with these characteristics

that, despite not being particularly awarded or widely known, receive a good reception from the audience. Patterns with more variables are less frequent but still significant.

The association rules were generated with confidence between 0.5 and 0.8, prioritizing moderately strong rules, with support greater than 0.1 to ensure sufficient representation in the dataset, and with lift greater than 1.5 indicating a stronger association between the antecedent and the consequent. The filtered rules were sorted in descending order of confidence. Here are some of the association rules:

N	Antecedents	Consequents	Support	Confidence	Lift
1	Multiple regions, High media content	many_votes	0.105106	0.744076	2.967454
2	many_votes, Multiple regions	High media content	0.105106	0.719883	2.786432
3	many_votes, High media content	Multiple regions	0.105106	0.708368	2.349927
4	many_votes	High media content	0.148378	0.591748	2.290460
5	many_votes	Multiple regions	0.146005	0.582282	1.931651

Table 5.2: Generated Association Rules

The table shows five association rules with antecedents and consequents. The support is low, ranging from 0.105 to 0.148, indicating that the combinations are not common but relevant. Confidence ranges from 0.582 to 0.744, with the strongest rule (0.744) linking "Multiple regions" and "High media content" to "many_votes". Lift is positive, ranging from 1.93 to 2.97, indicating a strong correlation between the antecedents and consequents. In general, the rules show that the combination of "many_votes" with "High media content" and "Multiple regions" has a significant relationship.

A rule-based model was created to predict the 'many_votes' column using two logical rules applied to the test dataset. The combination of these rules produces a final prediction, which was evaluated using a confusion matrix, accuracy, and a classification report. The model achieved an accuracy of 74.57%, with good performance on the 'False' class (precision: 86%, recall: 79%), but lower performance on the 'True' class (precision: 49%, recall: 61%), highlighting an imbalance between the classes. To improve the model, strategies such as addressing class imbalance, adding more complex rules, or testing advanced machine learning models can be explored.

Class	Precision	Recall
False	0.86	0.79
True	0.49	0.61
Accuracy	0.75	
Macro Avg	0.67	0.70
Weighted Avg	0.77	0.75

Table 5.3: Classification report for the rule-based model.

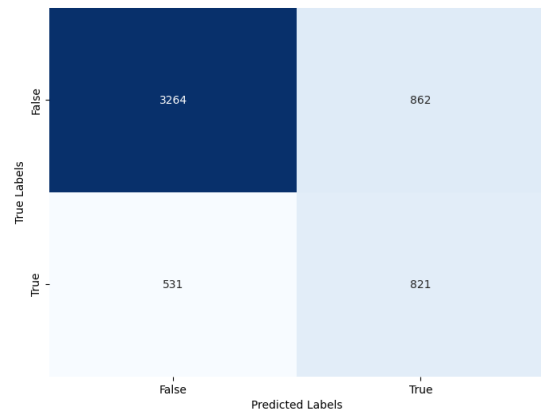


Figure 5.2: Confusion matrix